From Historical Pattern Search to Creative Infill: Designing Controllable Chinese Name Generation

Abstract

Neural Mingzi Generator began as a comparison between three name-generation models: a Markov baseline, an LSTM, and a small causal Transformer. The modeling study answers one question: how do recurrent and attention-based models behave on a short, constrained sequence task? The web app introduces a different question: what should a user-facing name generator do when a user already has a preferred character, surname, or style in mind?

This article describes the product and systems design behind the demo. The key decision is to separate **Historical Pattern** mode from **Creative Ancient-Style** mode. Historical Pattern mode asks what is supported by the cleaned historical-name corpus. Creative Ancient-Style mode asks the model to satisfy a user constraint while producing a plausible ancient-style name. Keeping those two modes separate makes the app more honest, more useful, and easier to evaluate.

1. Why Markov Still Matters

The Markov model is not just a weaker baseline beside neural models. It has product value because it is lightweight, interpretable, and easy to condition on subsets of the source data.

In this project, Markov generation serves three roles:

- **Baseline:** It provides a simple probabilistic reference point for the LSTM and Transformer.
- **Historical exploration:** It can be trained on dynasty-specific slices of CBDB-derived names, enabling a playful "Tang / Song / Ming / Qing" style mode when the private database is available.
- **Deployment fallback:** It is small enough to run reliably on low-memory public infrastructure, where neural checkpoints may exceed free-tier limits.

This is a useful engineering lesson: a baseline model can still be the right production surface for some user interactions, even if it is not the most expressive model in the research study.

2. The Product Problem: Fixed-Token Naming

Many users do not want a completely random name. They often have a character or phrase they already like, such as:

- a single character: 飞
- a low-frequency or semantically loaded character: 妃
- a two-character phrase: 若虚

At first, this sounds like one feature: "generate a name containing this token." In practice, it contains two different user intentions.

The first intention is historical:

> "Did historical Chinese names commonly use this character in this way?"

The second intention is creative:

> "Even if the historical corpus does not strongly support this character, can the model help me create a plausible ancient-style name?"

Those are different questions. A single endpoint should not pretend they are the same.

3. Historical Pattern Mode

Historical Pattern mode is evidence-oriented. It uses the cleaned historical-name corpus and constrained search to answer whether the requested token appears in stable naming patterns.

The interface asks for:

- fixed_token: one or two Chinese characters
- position: any, start, middle, or end
- optional seed: a surname or short prefix
- count: number of requested names

The backend returns not only generated names, but also corpus-support metadata:

- normalized fixed token
- support count in the cleaned corpus
- support level: strong, weak, or none
- sampling attempts
- whether search was exhausted
- an evidence message for the UI

The most important part is the failure language. If a token such as 妃 is weakly represented or absent in the cleaned name corpus, the app should not say:

> "Ancient Chinese people would not use this in a name."

That would overclaim. A better message is:

> "In the current cleaned historical-name corpus, this token does not show a stable observed naming pattern. It may be more common in other contexts, and this mode is evidence-limited."

That wording preserves the evidence boundary. It also tells the user what happened without turning a dataset limitation into a historical law.

4. Creative Ancient-Style Mode

Creative Ancient-Style mode serves a different purpose. It is not trying to reconstruct historical usage. It is trying to satisfy a user constraint while generating a name that feels compatible with the style of the training data.

The implementation uses a template-style fill-in-the-middle (FIM) route. Training examples are constructed from full names by extracting one- or two-character spans and asking a conditional decoder to reconstruct the full name from:

- task marker
- fixed token
- desired token position
- optional seed
- separator
- target full name

In this mode, a low historical support signal is not treated as a hard failure. Instead, the UI explicitly says that the output is creative generation:

> "This is Creative Ancient-Style mode. It satisfies the requested token where possible, but it should not be interpreted as historical reconstruction."

This distinction is small in the interface, but important in the product philosophy. The system can be both useful and honest if it labels the kind of answer it is giving.

5. Two Modes, Two Kinds of Truth

The split between Historical Pattern and Creative Ancient-Style mode reflects a broader ML product principle:

> A model should not only generate an answer. It should communicate what kind of answer it is giving.

Historical Pattern mode is conservative:

- stronger evidence boundary
- more explainable failures
- better for corpus-grounded claims
- lower coverage for rare or culturally complex tokens

Creative Ancient-Style mode is generative:

- higher coverage for user constraints
- better creative utility
- weaker historical claim
- requires clearer disclaimers

This is why the app keeps both. The right behavior is not always "generate at all costs." Sometimes the right behavior is "the historical evidence is weak, but you can switch to creative mode."

6. How This Fits the System

The public app has two major surfaces:

- **Playground:** live interaction with Classic generation, Compare, Dynasty-style Markov exploration, and Name Workshop.
- **Evaluation Lab:** lightweight feedback collection through blind pairwise tasks and dynasty-guess tasks.

The backend keeps live generation separate from formal evaluation:

- Playground requests call live inference endpoints.
- Evaluation Lab tasks are served from pre-generated task pools.
- Feedback is written through the FastAPI backend into managed Postgres/Supabase storage.
- The browser never receives database credentials.

That separation avoids mixing playful exploration with evidence collection. A favorite pick in Playground is useful product feedback, but it is not the same as blind evaluation evidence.

7. What This Adds Beyond the Model Study

The LSTM-vs-Transformer report focuses on optimization, parameter matching, and perplexity. This product design layer adds a different kind of engineering signal:

- Markov is treated as a useful baseline and deployment-friendly product component.
- FIM is used for controllable generation rather than general sampling.

- The UI separates historical plausibility from creative style transfer.
- Backend responses include support metadata instead of raw failure states.
- Feedback collection is routed through a server-side persistence layer.

For a portfolio project, this matters because it shows that the system is not only a collection of training scripts. It has model boundaries, API contracts, user-facing failure semantics, and a lightweight MLOps story.

Conclusion

The central design choice is simple:

> Historical evidence and creative generation should not share the same promise.

Historical Pattern mode answers: "What does the cleaned corpus support?" Creative Ancient-Style mode answers: "Can we satisfy this naming constraint in a plausible ancient style?"

Keeping those questions separate makes the project more trustworthy. It also turns a small name generator into a richer ML systems case study: a project about models, evidence boundaries, deployment constraints, and product judgment.